Recommendo: A Movie Recommendation WebApp

ML Engineer
Your Name

June 27, 2025

ABSTRACT

In this report, we present **Recommendo**, a machine learning-based web page for personalized movie recommendations. The site allows users to select 5 films they already enjoy and, through data-driven techniques, a recommendation engine computes movie similarities and generates personalized suggestions.

Introduction & Problem Statement

In a period in which a user has to choose between tens of different digital streaming platforms, they are overwhelmed by the sheer volume of available films and series. This has led to a sort of paradox of choice, in which these abundance of possibilities brought onto users decision fatigue.
Another keypoint is that, while there are tradition recommendation systems embedded directly into these services, they are not always transparent or personalized, particularly when users' history is not present.
Our proposal, Recommendo, has been built with these challenges in mind: we allow users to select up to 10 (at least one) films they already like and then return tailored recommendations based on those choices. The core of our project is a data-driven, machine learning-powered recommendation pipeline. My role as a machine learning engineer included think and program models, test their performances and work closer to the data engineer to make models more effective as they can.

Background

The development of our project required several things: we need to find coordination between modern web development practices, software design methodologies and data-driven machine learning components.
At the beginning of the project, we decided to adopt an Agile software development methodology: we broke down our work into weekly milestones and iterative improvements. Task tracking, collaboration and version control were managed using Github projects, which was used to structure the issues and pull request, milestones and so on. Each team member worked within their dedicated branch (e.g., dev-architecture for the SE, dev-ml for the MLE, dev-app for the SD and dev-data for the DE) and merged into the main branch only through reviewed pull request, reviewed by the software engineer.

Initially, we considered Streamlit for the frontend part, given its easy integration with Python-based ML models, which were the one we decided to work with. However, early in the development, the software developer brought up during a meeting that he felt it was too limited for the task at hand, and that he could do better if we transitioned to a more flexible stack: the Flask backend was built using Flask, to make the integration with python smoother while the frontend was built using HTML, CSS and JavaScript.
As the ML Engineer, I had to choose models that could work with minimal user input (1–10 film titles), offer low-latency responses, and integrate smoothly into a web-based pipeline. The models considered and/or developed were:

- k-NN on One-Hot Encoded features
- k-NN on Embedding-based features (SBERT)
- VAE (Variational Autoencoder) on Sentence Embeddings – later discarded due to poor performance

The dataset used was a public Netflix dataset from Kaggle. It contains key metadata such as title, cast, genre, description, and rating. Preprocessing was a critical step to ensure data was in a form suitable for both classic ML and embedding-based models.
The dataset management and preprocessing were handled by the data engineer, while their integration into the Flask backend required coordination with the software developer. The dataset we decided to use was a dataset containing Netflix's Movies and TV-series, found on Kaggle.

Methodology/Approach

Machine Learning Pipeline

The recommendation system was developed through a structured pipeline:

1. **Data Preprocessing**:
   Collaborated with the Data Engineer to handle heterogeneous features:
   - Categorical variables (`director, country, cast`) one-hot encoded
   - The one-hot encoded high-cardinality features (`cast`) filtered to actors appearing in ≥ 5 productions
   - Text features (`description, listed_in, cast, title`) processed with Sentence-BERT embeddings

- `duration` converted to numerical format
- Normalized numerical variables `release_year`, `rating`
- Dropped non intersting variables (`index`, `date_added`)
- Dropped observation with missing values on variables different from `director`, `cast`, `country`

2. **Feature Engineering**:
   Created two distinct feature spaces:
   - *Sparse Representation*: Traditional one-hot encoding of metadata
   - *Dense Representation*: SBERT embeddings (`all-MiniLM-L6-v2`) of textual descriptions

   Assert meaningful vector representation on embedded features using `t-SNE` and `UMAP`

3. **Model Development**:
   Implemented and evaluated three approaches:
   - *KNN (One-Hot)*: Traditional k-Nearest Neighbors using mean squared error on sparse features
   - *KNN (Embedded)*: Cosine similarity in SBERT embedding space
   - *VAE*: Variational Autoencoder for latent space learning (abandoned due to poor cluster separation on the latent space)

   On KNN models an hyperparameter for weight the relevance of features has been added, which enhance model performance by given less importance to feature like `Title` and more others like the movie genre.

4. **Model Integration**:
   Designed unified load and prediction interface:

```
def load(self):
    """load necessary data"""


def predict(IDX: list, k=5) -> np.ndarray:
    """Returns top-k similar
        movie indices"""
```

   Designed similar `fit()` interface to compute the embedding and to store the data

   Collabarated with the Data Engineer to fine tune the hyperparameters.

MLOps Implementation

The machine learning lifecycle followed MLOps principles:

- **Version Control**: Models, datasets, and preprocessing pipelines versioned in GitHub
- **Reproducibility**: Frozen dependencies via requirements.txt and model serialization
- **Testing**: Unit tests for embedding consistency and prediction stability
- **Monitoring**: Latency tracking during development (sub-second response achieved)

Results

Performance Evaluation

Models were evaluated on four critical dimensions:

Metric	🟦	🟩
Inference Speed	332 ms	232 ms
Memory Footprint	82 MB	62 MB
Cluster Coherence†	low	high
Human Evaluation†	medium	high

Table 1: 🟦 KNN (One-Hot) 🟩 KNN (Embedded)
†Human evaluation: 4 users rated relevance (4 test users)

Key Findings

The KNN (Embedded) model demonstrated superior performance across all evaluation metrics:

- Achieved 30% **faster inference** by avoiding high-dimensional computations
- Enabled semantic analysis of textual features `description` and `Title`
- Produced more semantically coherent recommendations (validated via `t-SNE` and `UMAP` visualizations)
- Successfully integrated with backend through standardized interface
- Dimensionality of features:
  - KNN (one-hot): 2356 features (high cardinality from one-hot encoding)
  - KNN (Embedded): 9 features, 6 SBERT with vector representation of 384 elemets + 3 normalized numerical, for a total of 2307 numerical values per observation

System Outcomes

- Enabled for both models real-time suggestions ($<$ 500 ms response time)
- Implemented model-agnostic API supporting runtime switching: Designed polymorphic interface enabling runtime model switching
  → *Implementation*: Standardized `predict()` method across models
  → *Benefit*: Frontend can toggle between KNN/Embedded models via dropdown

Discussion & Conclusion

Technical Challenges

Several challenges emerged during development:

- **Heterogeneous Data**: TV shows and movies required different duration handling
- **Semantic Gaps**: Plot descriptions often lacked thematic depth for accurate embedding; more detailed description would lead to a better results on the KNN Embedded model.
- **VAE Limitations**: Failed to learn meaningful latent representations from sparse features
- **Cold Start**: New movies without sufficient metadata remain challenging

System Limitations

The current implementation has several constraints:

- Static dataset (Netflix catalog as of 2021)
- No personalization beyond initial seed movies
- Poor movie descriptions

Conclusion

The developed recommendation system successfully addresses the core requirement of generating personalized movie suggestions from minimal user input. The KNN approach using sentence embeddings proved optimal, balancing performance with recommendation quality. The modular design allows seamless incorporation of future improvements.

Future Enhancements

- Implement ensemble approach combining multiple models
- Incorporate lightweight user feedback mechanism
- Develop genre-aware similarity adjustments
- Incorporate external APIs for richer metadata
- Implement user feedback mechanism to improve recommendations

The project demonstrates that content-based filtering with modern NLP techniques provides effective recommendations while meeting strict performance constraints required for web deployment. The MLOps approach ensured smooth integration between machine learning components and production web environment.